

バグ修正報告書

InterviewResult#rejected? テスト失敗の原因分析と修正

プロジェクト	AI面接インタビューシステム
フレームワーク	Ruby on Rails 6.1.7 / Ruby 3.1.6
報告日	2026年3月21日
重要度	Medium
ステータス	修正済み
影響範囲	InterviewResult モデル — 不合格判定ロジック

1. 障害概要

`bundle exec rspec` 実行時、全254テスト中1件が失敗。

項目	内容
テスト ファイル	<code>spec/models/interview_result_spec.rb:52</code>
テスト 名	<code>InterviewResult#rejected? returns false when rejection_details empty</code>
期待値	<code>false</code>
実測値	<code>true</code>
テスト 総数	254 examples, 1 failure

エラー出力

```
Failures:

  1) InterviewResult#rejected? returns false when rejection_details is empty
     Failure/Error: expect(result.rejected?).to be false

     expected false
     got true
     # ./spec/models/interview_result_spec.rb:54:in `block (3 levels) in <anonymous>'
```

2. 原因分析

2.1 修正前コード

ファイル: `app/models/interview_result.rb`

```
def rejected?
  rejection_details.present? && rejection_details != {}
end
```

2.2 根本原因：SQLite の JSON カラム型の挙動

DBスキーマでは `rejection_details` カラムは以下のように定義されています:

```
t.json "rejection_details", default: {}
```

ここで問題となるのが **SQLite の JSON カラムの内部表現** です。

【PostgreSQL（本番環境）の場合】

DB格納値: <code>{}</code>	(ネイティブ JSON型)
Rails読取: <code>{}</code>	(Ruby Hash)
<code>{}.present? → false</code>	✅ 正常動作

【SQLite（開発/テスト環境）の場合】

DB格納値: <code>"{}"</code>	(TEXT型として格納)
--------------------------	--------------

Rails読取: "{}" (Ruby String)
"{}".present? → true ❌ 意図しない動作

原因まとめ: SQLite は JSON 型をネイティブサポートしておらず、TEXT 型として格納します。Rails が値を読み取ると Ruby の **String** "{}" として返されます。String#present? は空文字でない限り true を返すため、空ハッシュであるはずの rejection_details が「存在する」と誤判定されました。

2.3 比較表

チェック式	PostgreSQL ({} Hash)	SQLite ("{}" String)
.present?	false ✓	true ❌
!= {}	false ✓	true ❌
.blank?	true ✓	false ❌

つまり修正前のコードは **PostgreSQL 前提** であり、SQLite 環境では常に true を返してしまう問題がありました。

3. 修正内容

3.1 修正後コード

ファイル: app/models/interview_result.rb:35-40

```
def rejected?  
  details = rejection_details  
  return false if details.blank?  
  
  parsed = details.is_a?(String) ? (JSON.parse(details) rescue nil)  
  parsed.is_a?(Hash) && parsed.any?  
end
```

3.2 修正ロジックの説明

ステップ	処理内容	目的
1	<code>details.blank?</code> チェック	<code>nil</code> ・ 空文字 ・ 空ハッシュを早期リターン
2	<code>is_a?(String)</code> 判定	SQLite が返す文字列型を検出
3	<code>JSON.parse</code> + <code>rescue</code>	文字列を安全にHashへ変換
4	<code>parsed.is_a?(Hash) && parsed.any?</code>	中身のあるHashの場合のみ <code>true</code>

3.3 各入力パターンの動作確認

入力値	型	修正前	修正後
<code>nil</code>	NilClass	<code>false</code>	<code>false</code> ✓
<code>{}</code>	Hash	<code>false</code>	<code>false</code> ✓
<code>"{}"</code>	String	<code>true</code> ✗	<code>false</code> ✓
<code>{"reason"=>"スコア不足"}</code>	Hash	<code>true</code>	<code>true</code> ✓
<code>'{"reason":"スコア不足"}'</code>	String	<code>true</code>	<code>true</code> ✓

4. テスト実行結果

4.1 対象テスト単体実行

```
$ bundle exec rspec spec/models/interview_result_spec.rb:52

InterviewResult
  #rejected?
    returns false when rejection_details empty

1 example, 0 failures
```

4.2 InterviewResult 全テスト実行

```
$ bundle exec rspec spec/models/interview_result_spec.rb

InterviewResult
  #rejected?
    returns true when rejection_details present
    returns false when rejection_details empty
  uniqueness
    prevents duplicate results for same interview
  associations
    is expected to belong to interview required: true
  validations
    is expected to validate that :interview_id cannot be empty/false
  #completion_percentage
    returns 0 when total_questions is zero
    calculates correct percentage
    returns 0 when total_questions is nil
  enums
    is expected to define :final_status as an enum backed by an integer
  results_data store
    stores and retrieves values

10 examples, 0 failures
```

結果: 全10テストが合格。修正前に失敗していたテストケースが正常に動作することを確認しました。

5. 影響範囲と今後の注意事項

5.1 影響範囲

対象	影響
本番環境 (PostgreSQL)	影響なし — PostgreSQL はネイティブ JSON 型を使用するため、修正前でも正常動作
開発/テスト環境 (SQLite)	修正により正常動作 — 文字列型の JSON 値を正しく処理

面接結果の不合格判定	SQLite 環境で「全結果が不合格」と誤表示されていた可能性を解消
------------	------------------------------------

5.2 今後の注意事項

- 1. **DB間の型差異に注意:** SQLite と PostgreSQL で `json` カラムの挙動が異なるため、JSON カラムを扱うメソッドでは型チェックを行うこと
- 2. **開発/テスト環境の統一検討:** 本番と同じ PostgreSQL を開発環境でも使用することで、この種の不整合を根本的に防止可能
- 3. **類似パターンの確認:** 他のモデルで JSON カラムのデフォルト値 `{}` を `present?` で判定している箇所がないか定期的にチェック

6. 変更ファイル一覧

ファイル	変更種別	変更行
<code>app/models/interview_result.rb</code>	バグ修正	35〜40行目 (<code>rejected?</code> メソッド)